

MSc Project Plan

Student Name	Haoxuan Cheng	Student ID	20621569
Course	MSc CS and AI	Supervisor	Colin Johnson
Module Code	COMP4026	Word Count	Approx. 970 excluding figures, tables and references

1. Project Title

Mental Imagery and Programming Ability: Visual Scaffolding for Code Tracing Across Different Imagery Profiles

2. Project Overview

This project investigates the relationship between mental imagery and programming ability by focusing on code tracing: the ability to predict how variables, control flow and program state change during execution. Code tracing is a suitable operationalization of programming ability because it is measurable, technically meaningful and closely linked to program comprehension. The project does not assume that visualization is automatically better than text. Instead, it asks whether participants with different imagery profiles benefit differently from targeted visual scaffolding, such as variable-state tables, execution timelines and control-flow cues.

The project will include a substantive programming component. A full-stack experimental platform will be designed to deliver imagery-profile questions, code-tracing tasks and scaffolded visual representations; record answers, timing, confidence and optional explanations; and export structured data for analysis. The study will recruit computing students or participants with programming experience, not only beginners. The contribution is therefore both technical and empirical: a research instrument for controlled code-tracing tasks, a data-analysis pipeline, and evidence-informed discussion of how imagery-related individual differences interact with visual support in programming tasks.

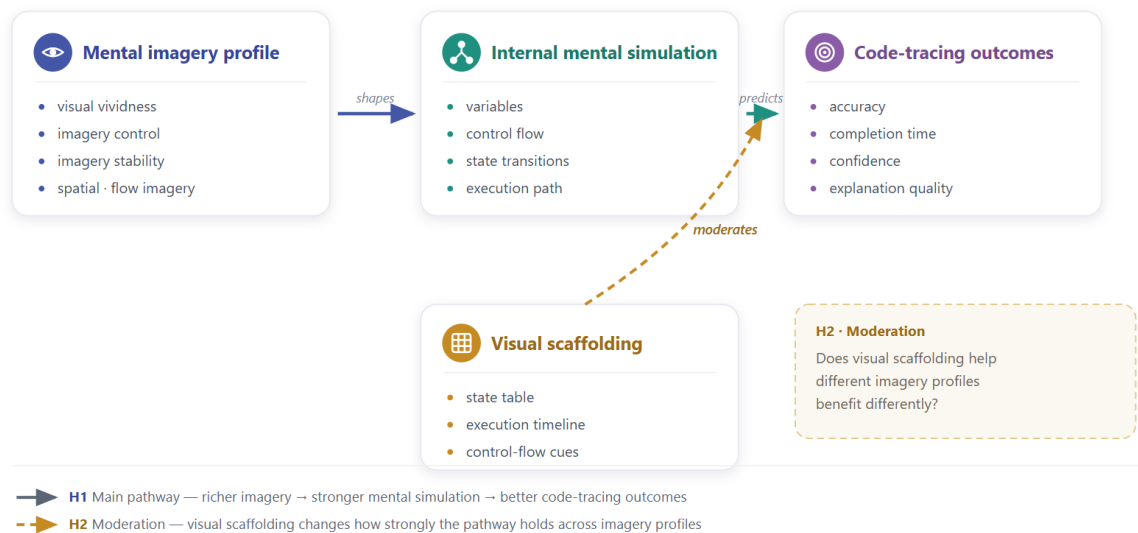


Figure 1. Conceptual model linking imagery profile, mental simulation, visual scaffolding and code-tracing outcomes.

3. Related Work

Existing research related to this project can be grouped into three areas. First, mental imagery research provides a basis for measuring individual differences. Marks (1973) showed that vivid visual imagery is associated with more accurate picture recall, while Andrade et al. (2014) developed the Plymouth Sensory Imagery Questionnaire to measure imagery across several sensory modalities. Pearson (2019) further explains that visual mental imagery can operate like a weaker form of perception and varies substantially between individuals. These studies support the idea that imagery should be treated as a measurable cognitive profile rather than a simple visual/non-visual ability.

Second, programming research shows that mental representation is central to program understanding. Petre and Blackwell (1999) found that expert programmers use dynamic imagery such as mental machines, landscapes and moving symbolic structures when reasoning about software. Petre (2010) further shows that expert teams externalize such imagery through sketches, metaphors and custom visualizations. Heinonen et al. (2023) synthesize research on programmers' mental models and argue that these models guide comprehension and task performance, but that the field remains conceptually fragmented.

Third, software and program visualization research has produced taxonomies, visual programming environments, algorithm animations and educational program visualization systems (Sorva et al., 2013). However, existing visualizations often focus on presenting program behavior in general, rather than adapting support to individual cognitive differences. Naps et al. (2002) also argue that visualization is most effective when learners actively engage with it. This project builds on these studies by designing a task-based platform that combines imagery profiling, code-tracing tasks, visual scaffolds and performance analytics, allowing the relationship between imagery profiles and programming performance to be examined more clearly.

4. Methodology

The project will use a combined system-development and empirical-evaluation methodology. First, the literature review will be used to define the mental imagery dimensions, code-tracing task types and visual-scaffolding conditions. Second, a full-stack experimental platform will be implemented. The frontend will use React, TypeScript and Vite for modular questionnaire screens, task presentation and interaction control. A custom TypeScript task engine, informed by behavioral experiment platforms such as jsPsych, will manage task sequencing, condition assignment, timing and response capture. Visual scaffolds will be implemented using SVG and D3.js, because the project requires custom representations of program state, execution timelines and control-flow transitions rather than only standard charts.

The backend will use Python FastAPI to receive participant responses, validate task logs and export a structured performance dataset. SQLite will be used during development, with PostgreSQL considered if deployment requires multi-user data storage. Data analysis will be conducted in Python using pandas and statistical libraries, with matplotlib or plotly used for result visualization. The study will compare code-only and scaffolded tracing tasks. Evaluation measures will include accuracy, completion time, confidence, perceived workload and explanation quality. The results will be evaluated by examining performance patterns across imagery dimensions and programming experience, using descriptive statistics and correlation or group-comparison methods where the sample size allows.

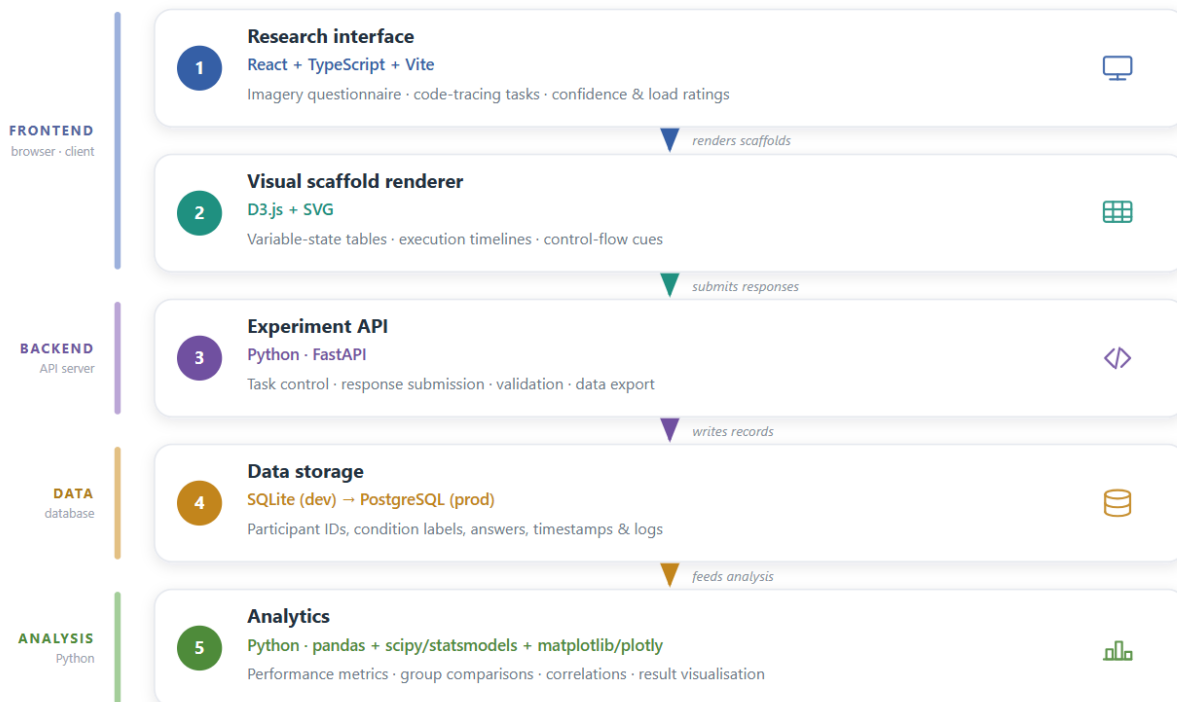


Figure 2. Proposed technical architecture for task delivery, visual scaffolding, data logging and analytics.

5. Programme of Work

WP1 - Literature and construct definition

Define imagery dimensions, code-tracing outcomes, scaffolding types and programming-experience variables.

M1: Research model and requirements completed.

WP2 - Task and scaffold design

Create code-only and scaffolded tracing tasks involving variables, loops, conditionals, functions and state changes.

M2: Pilot-ready task set and scaffold designs completed.

WP3 - Experimental platform implementation

Develop the React/TypeScript interface, D3/SVG scaffolds, FastAPI backend, database schema and export functions.

M3: Functional platform with data logging.

WP4 - Pilot test and refinement

Run pilot sessions to check usability, timing accuracy, task difficulty and data completeness.

M4: Refined instrument ready for study.

WP5 - User study and data collection

Recruit participants with programming experience and collect performance, confidence and workload data.

M5: Dataset completed.

WP6 - Data analysis and dissertation

Analyze results, generate visualizations, interpret findings and write the final dissertation.

M6: Final analysis and dissertation completed.

6. Project Timeline

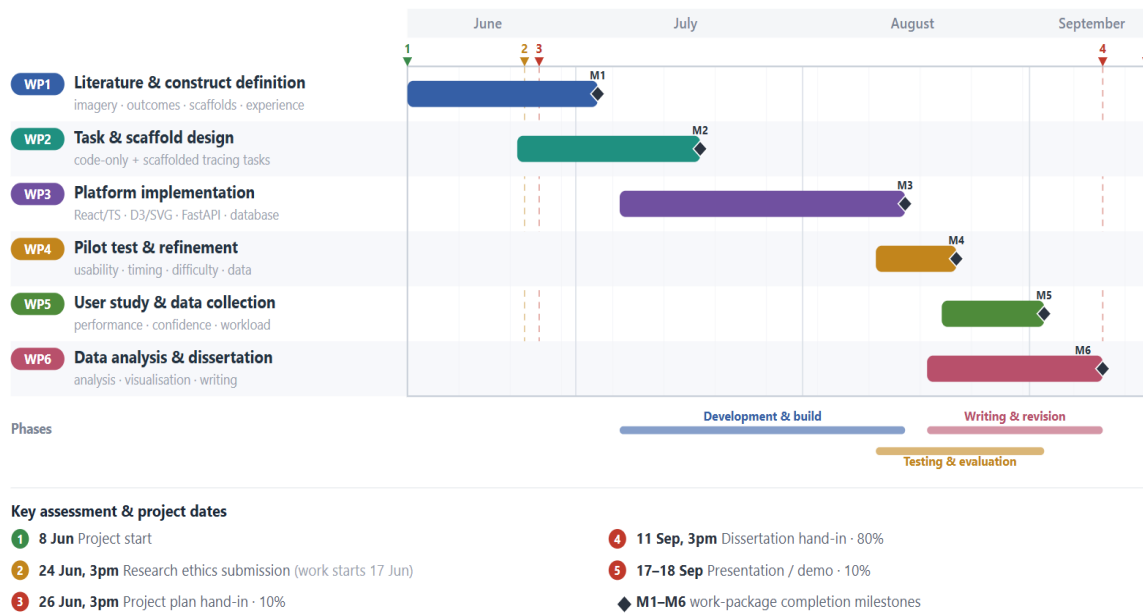


Figure 3. 14-week project timeline with milestones.

7. Risk Management

The main project risk is that the technical implementation may become too broad. This will be managed by prioritizing the core functions: task delivery, visual scaffolding, response logging and data export. A second risk is limited participant recruitment, which will be mitigated by using a feasible sample of computing students or participants with programming experience. A third risk is that the sample size may not support strong statistical claims; therefore, the analysis will combine descriptive statistics, visual analysis and cautious interpretation. Data quality risks will be reduced through pilot testing, clear task instructions and validation of response logs before the main study.

Additional Information

References

- [1] Andrade, J., May, J., Deeprose, C., Baugh, S.-J., & Ganis, G. (2014). Assessing vividness of mental imagery: The Plymouth Sensory Imagery Questionnaire. *British Journal of Psychology*, 105(4), 547–563. <https://doi.org/10.1111/bjop.12050>
- [2] Heinonen, A., Lehtelä, B., Hellas, A., & Fagerholm, F. (2023). Synthesizing research on programmers’ mental models of programs, tasks and concepts — A systematic literature review. *Information and Software Technology*, 164, 107300. <https://doi.org/10.1016/j.infsof.2023.107300>

[3] Marks, D. F. (1973). Visual imagery differences in the recall of pictures. *British Journal of Psychology*, 64(1), 17–24. <https://doi.org/10.1111/j.2044-8295.1973.tb01322.x>

[4] Naps, T. L., Rößling, G., Almstrum, V., Dann, W., Fleischer, R., Hundhausen, C., Korhonen, A., Malmi, L., McNally, M., Rodger, S., & Velázquez-Iturbide, J. Á. (2002). Exploring the role of visualization and engagement in computer science education. *ACM SIGCSE Bulletin*, 35(2), 131–152. <https://doi.org/10.1145/782941.782998>

[5] Pearson, J. (2019). The human imagination: The cognitive neuroscience of visual mental imagery. *Nature Reviews Neuroscience*, 20, 624–634. <https://doi.org/10.1038/s41583-019-0202-9>

[6] Petre, M., & Blackwell, A. F. (1999). Mental imagery in program design and visual programming. *International Journal of Human-Computer Studies*, 51(1), 7–30. <https://doi.org/10.1006/ijhc.1999.0267>

[7] Petre, M. (2010). Mental imagery and software visualization in high-performance software development teams. *Journal of Visual Languages & Computing*, 21(3), 171–183. <https://doi.org/10.1016/j.jvlc.2009.11.001>

[8] Sorva, J., Karavirta, V., & Malmi, L. (2013). A review of generic program visualization systems for introductory programming education. *ACM Transactions on Computing Education*, 13(4), Article 15. <https://doi.org/10.1145/2490822>